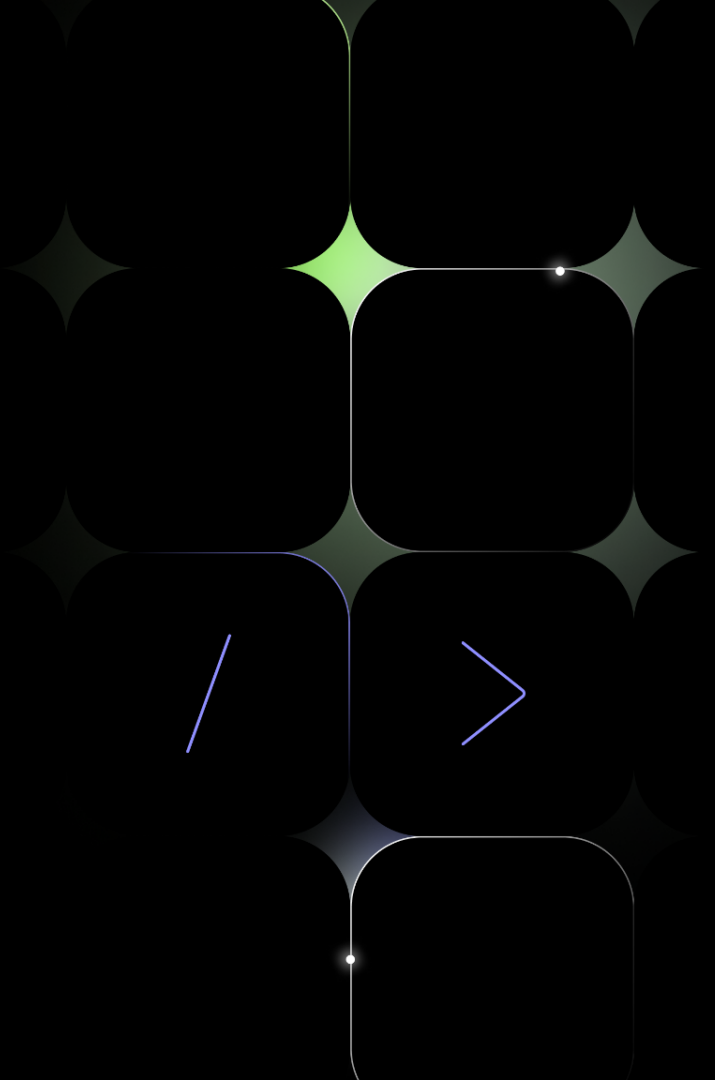
monday AI engineering

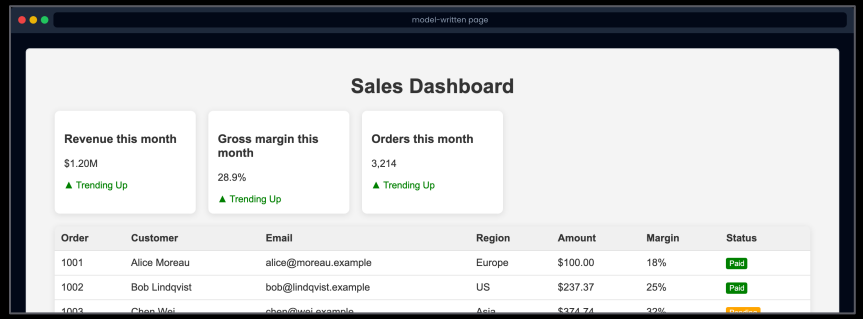
Generative UI for Production

How LLMs compose interfaces within your design system

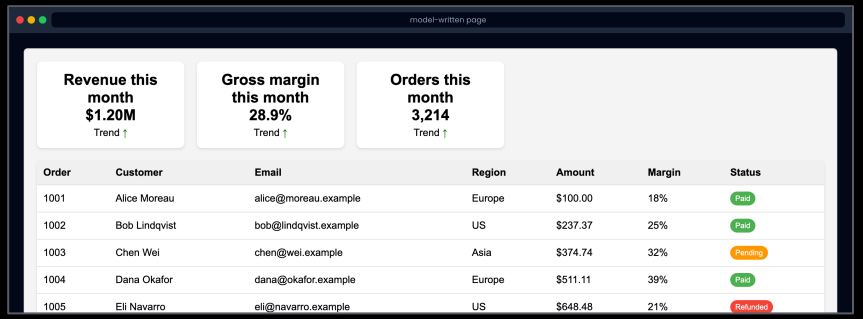
Konstantinos Leimonis
Tech Lead, Client Foundations, monday.com
CityJS · 23 Oct 2026



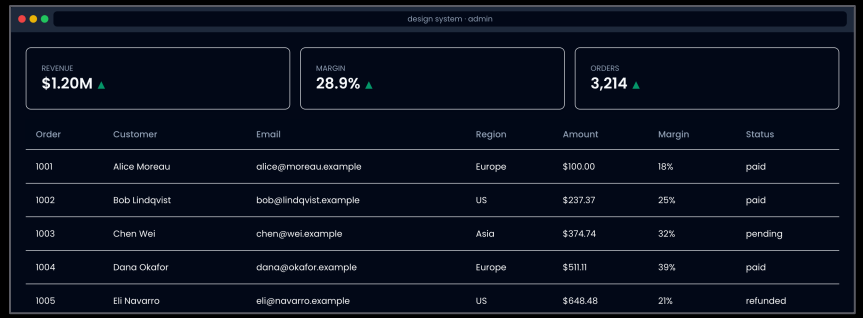
› Build me a sales dashboard.



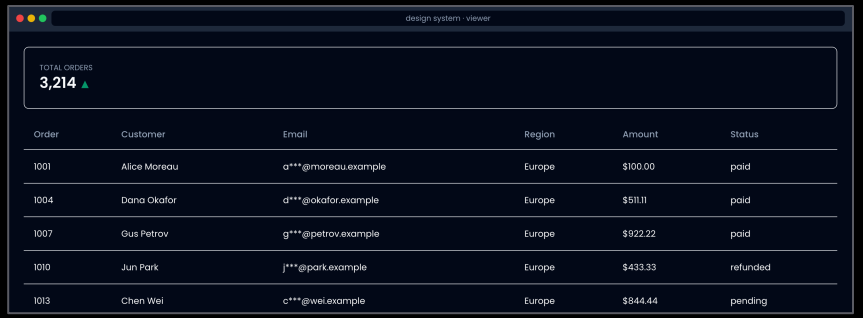
model-written page · run 1



model-written page · run 2



design system · admin



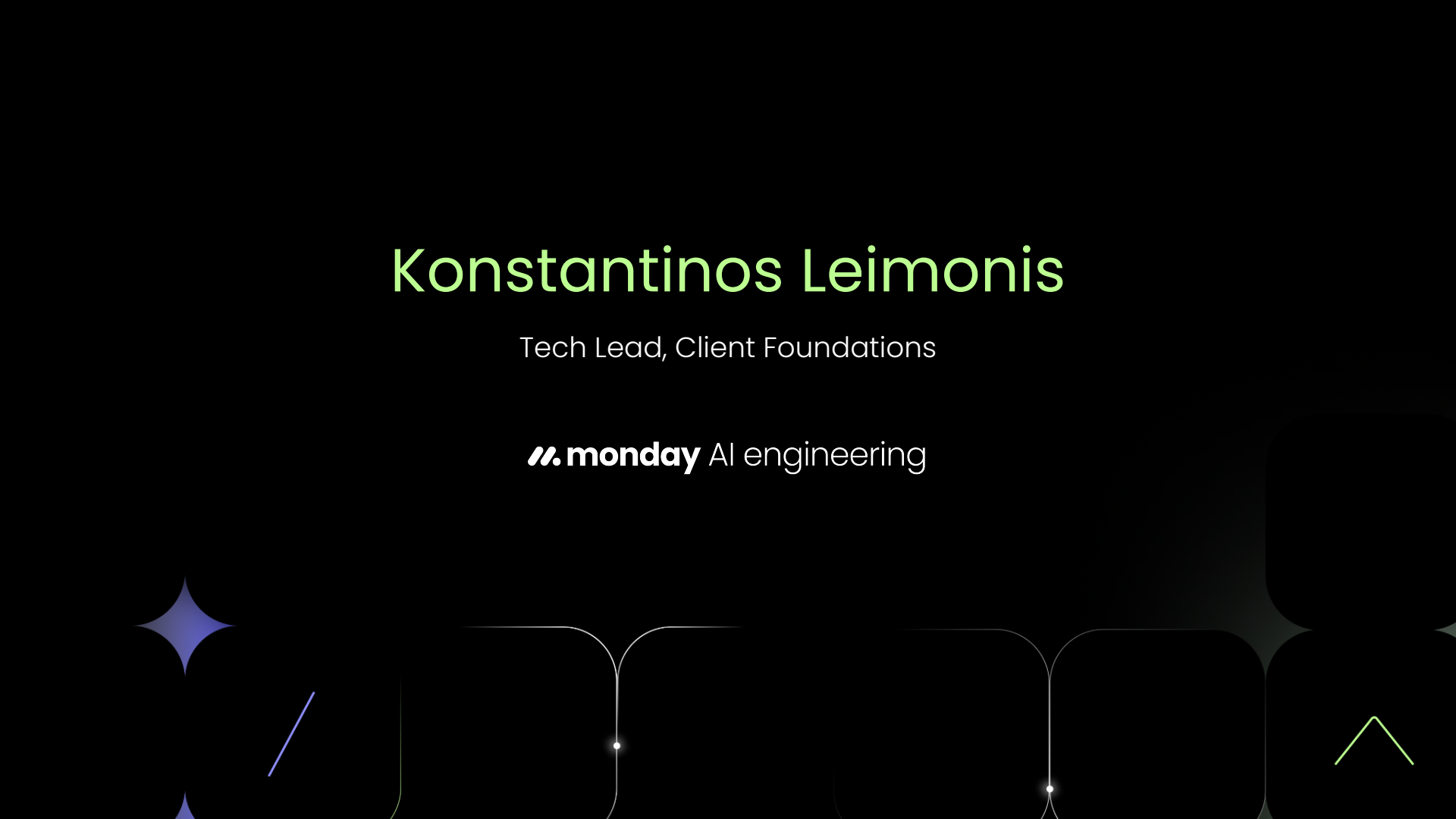
design system · viewer

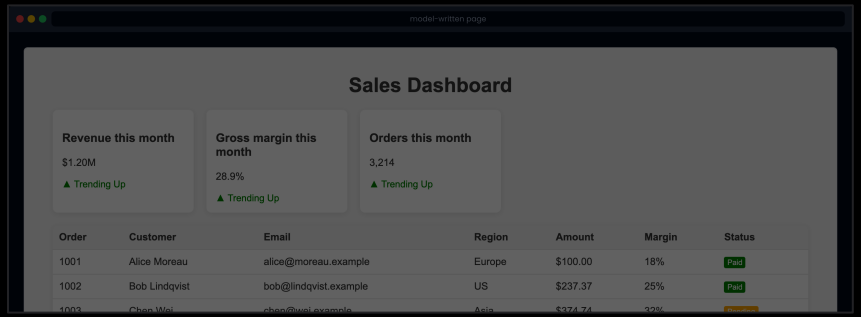
Same prompt. Same model. Three different products.

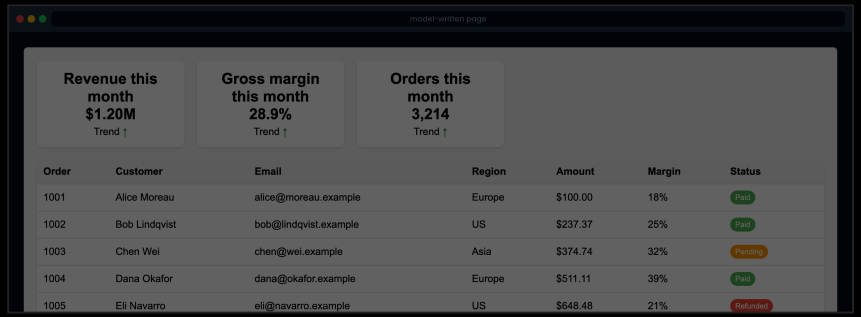
Konstantinos Leimonis

Tech Lead, Client Foundations

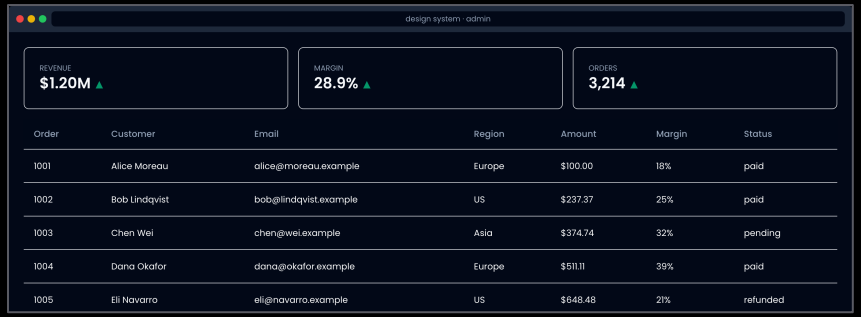
monday AI engineering

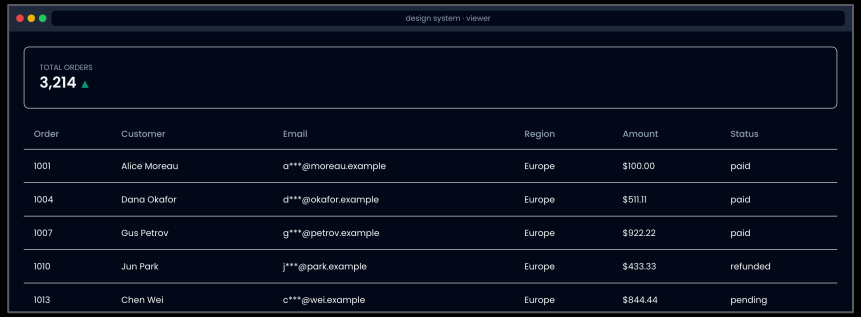






every demo you've seen

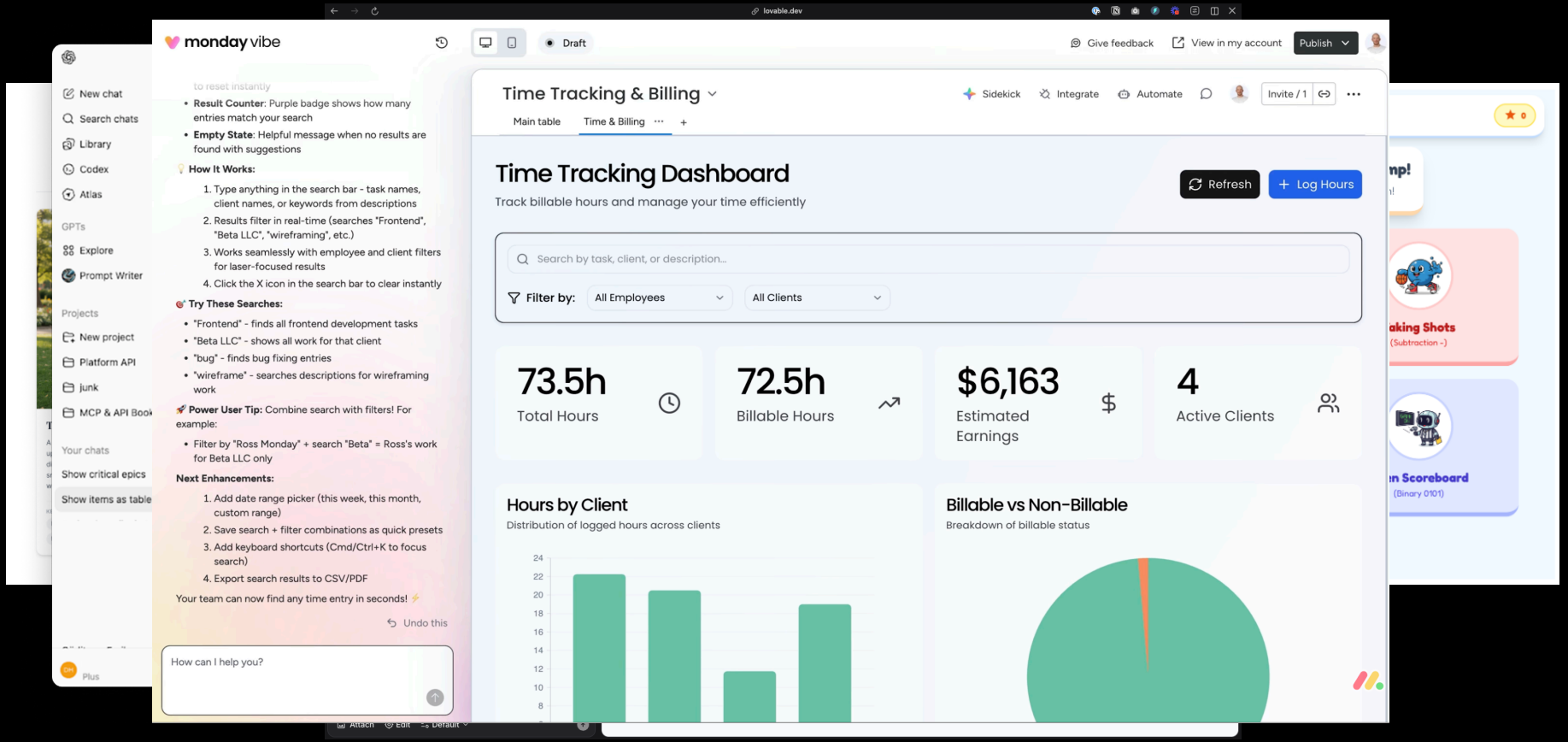




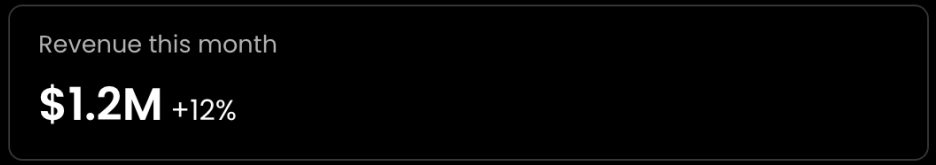
this talk

1. Design system?
2. A model renders a component?
3. Changes by who's signed in?

Already in products



user: Show me revenue this month.
assistant:




```
show me revenue this month
```

How it works



name · description · schema

The tool

server

```
export const showMetricTile = tool({
  // What the model reads to decide whether this is the right tool: the routing.
  description: "Show the revenue tile for a period.",
  // The only thing the model fills in.
  inputSchema: z.object({
    period: z.enum(["today", "this_week", "this_month", "this_quarter"]),
  }),
  // Where the numbers come from. The model never writes one.
  execute: async ({ period }) => METRICS[period].revenue,
});
```

description

inputSchema

execute

The message switch

client

```tsx
export function Message({ message }: { message: UIMessage["parts"][number] }) {
  if (message.type === "text") return <span>{message.text}</span>;

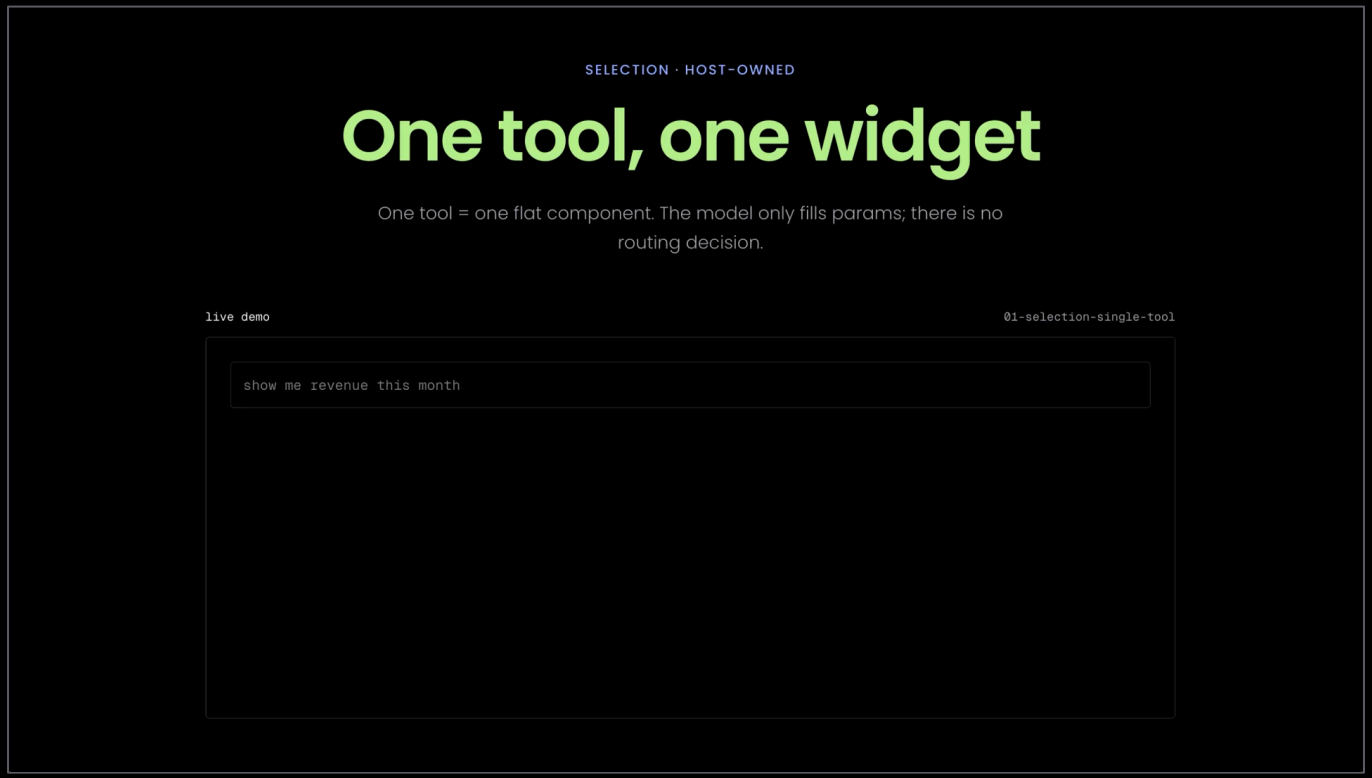
  if (message.type === "tool-showMetricTile") {
    if (message.state === "input-available") return <MetricTileSkeleton />;
    if (message.state === "output-available") {
      const output = message.output as { label: string; value: string; delta: string };
      return <MetricTile {...output} />;
    }
  }
  return null;
}
```

input-available → skeleton

output-available → tile

Ask for revenue, get a tile

live



Four tools

server

```ts
const period = z.enum(["today", "this_week", "this_month", "this_quarter"]);
const region = z.enum(REGIONS as [Region, ...Region[]]);
const executeFn = async () => ({}); // real bodies read lib/data.ts

export const tools = {
  showMetricTile: tool({
    description: "Show one headline metric tile (revenue, margin or order count) for a period.",
    inputSchema: z.object({ metric: z.enum(["revenue", "margin", "orders"]), period }),
    execute: executeFn,
  }),
  showOrdersTable: tool({
    description: "Show a table of recent orders, optionally filtered to one region or one status.",
    inputSchema: z.object({ region: region.nullable(), status: z.enum(["paid", "pending",
        "refunded"]).nullable() }),
    execute: executeFn,
  }),
  showApprovalsQueue: tool({
    description: "Show the queue of refunds, discounts and credit notes waiting for a manager's approval.",
    inputSchema: z.object({}),
    execute: executeFn,
  }),
  showRegionBreakdown: tool({
    description: "Show revenue per region (Europe, US, Asia) as a bar list, for comparing regions.",
    inputSchema: z.object({}),
    execute: executeFn,
  }),
};
```

Orders in Europe Example

› Show me recent orders in Europe. → `showOrdersTable · region: "Europe"`

user:
Show me revenue this month.
assistant:

Revenue this month

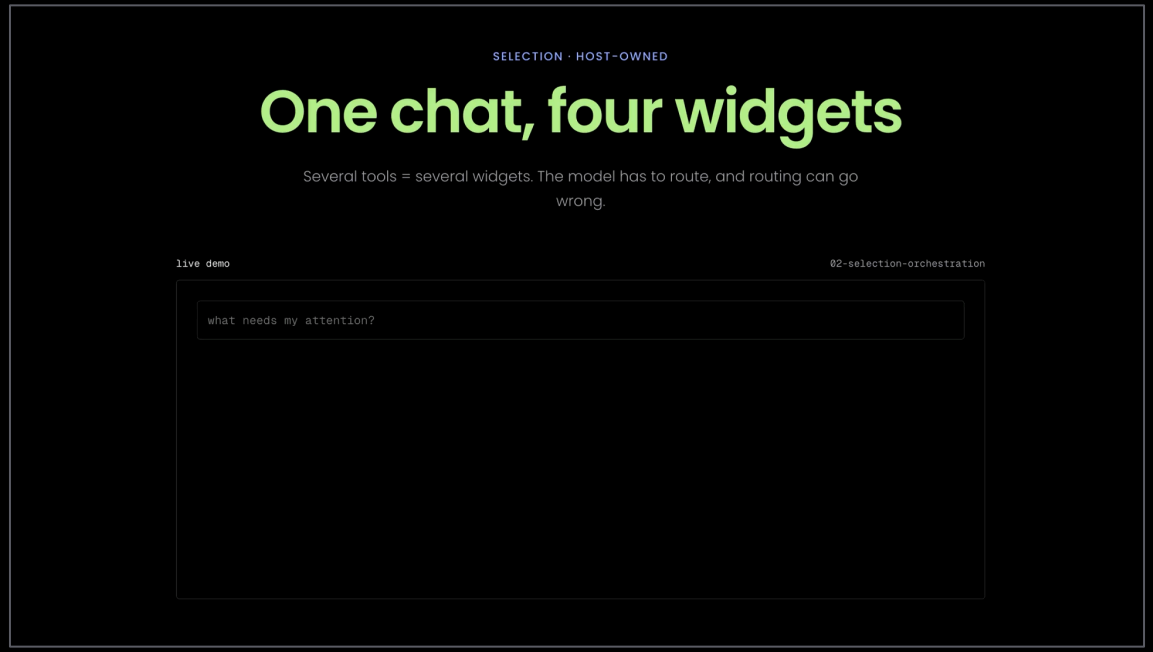
$1.2M +12%

user:
Show me recent orders in Europe.
assistant:

Order	Customer	Region	Amount	Status
1001	Alice Moreau	Europe	$100.00	paid
1004	Dana Okafor	Europe	$511.11	paid
1007	Gus Petrov	Europe	$922.22	paid

How is margin looking in the US?

› How is margin looking in the US? → global margin tile



showMetricTile inputSchema

server

```
// "How is margin looking in the US?" -> showMetricTile({ metric: "margin", period }).
// There is no region field, so the US can't be said. The model drops it silently.
const period = z.enum(["today", "this_week", "this_month", "this_quarter"]);

export const showMetricTile = tool({
  description: "Show one headline metric tile (revenue, margin or order count) for a period.",
  inputSchema: z.object({ metric: z.enum(["revenue", "margin", "orders"]), period }), // no region
  execute: async ({ metric, period }) => METRICS[period][metric],
});
```

no region

the tool list is the limit

One argument added

server

```
// One argument added. "How is margin looking in the US?" can now be said.
const period = z.enum(["today", "this_week", "this_month", "this_quarter"]);
const region = z.enum(REGIONS as [Region, ...Region[]]);
const input = z.object({
  metric: z.enum(["revenue", "margin", "orders"]),
  period,
  region: region.nullable(), // + one argument
});

export const showMetricTile = tool({
  description:
      "Show one headline metric tile (revenue, margin or order count) for a period, optionally for one region.",
  inputSchema: input,
  execute: async ({ metric, period, region }) => (region ? regionMetric(metric,
      region) : METRICS[period][metric]),
});

// The call the model makes now.
export const call: z.infer<typeof input> = { metric: "margin", period: "this_month", region: "US" };
```

one argument added

now it can be asked

MetricTile w/ our Design System

```
--- 02-selection-orchestration/app/components.tsx
+++ 02b-selection-design-system/app/components.tsx
@@ -1,8 +1,16 @@
 export function MetricTile({ label, value, delta }: Metric) {
+  const up = !delta.startsWith("-");
   return (
-    <div className="widget">
-      <div className="widget-label">{label}</div>
-      <strong className="widget-value">{value}</strong> {delta}
-    </div>
+    <Card className={up ? "border-l-4 border-l-emerald-500" :
+        "border-l-4 border-l-rose-500"}>
+      <CardHeader>
+        <CardTitle>{label}</CardTitle>
+      </CardHeader>
+      <CardContent>
+        <span className="text-4xl font-bold">{value}</span>
+        <Badge className={up ?
+            "bg-emerald-500/15 text-emerald-400" :
+            "bg-rose-500/15 text-rose-400"}>
+          {up ? "▲" : "▼"} {delta}
+        </Badge>
+      </CardContent>
+    </Card>
   );
 }
```

model output: unchanged

your component: swapped

– removed + added

Revenue this month

$1.2M +12%

02 · plain

REVENUE THIS MONTH

$1.2M ▲ +12%

02b · design system

Airbnb Ghost Platform

sections and screens · 2021

```
export const ghostPlatformResponse = {
  sections: [{ id: "title",
      sectionComponentType: "TITLE",
      section: { title: "Revenue this month"
      } }],
  screens: [{ id: "root",
      layout: { placements: [{ sectionId:
      "title" }] } }],
};
```

Microsoft Adaptive Cards

JSON card · docs from 2017

```
export const adaptiveCard = {
  type: "AdaptiveCard",
  version: "1.5",
  body: [
    { type: "TextBlock",
        text: "Revenue this month",
        weight: "Bolder" },
    { type: "FactSet",
        facts: [{ title: "Revenue",
        value: "$1.2M" }, { title: "Change",
        value: "+12%" }] },
  ],
};
```

Catalog excerpt

your design system, described once

```
export const catalog = defineCatalog(schema, {
  components: {
    StatTile: {
      description: "One headline metric. `value` MUST be {\"$state\": \"/data/metrics/<name>/value\"}, never literals.",
      props: z.object({
        label: z.string(),
        value: z.union([z.string(), z.number()]),
        trend: z.enum(["up", "down", "flat"]).nullable(),
      }),
    },
    Table: {
      description: "Tabular data. `columns` and `rows` MUST be $state references.",
      props: z.object({ columns: z.array(z.string()).min(1), rows: z.array(z.array(z.string())) }),
    },
    ApprovalsPanel: {
      description: "Side panel listing refunds/discounts awaiting approval.",
      props: z.object({
        title: z.string(),
        items: z.array(z.object({ id: z.string(), type: z.string(), orderId: z.string(), requester: z.string(),
            amount: z.string() })),
      }),
    },
  },
  actions: {
    refresh_data: { description: "Reload the figures shown on the page." },
  },
});
```

Three JSONL lines

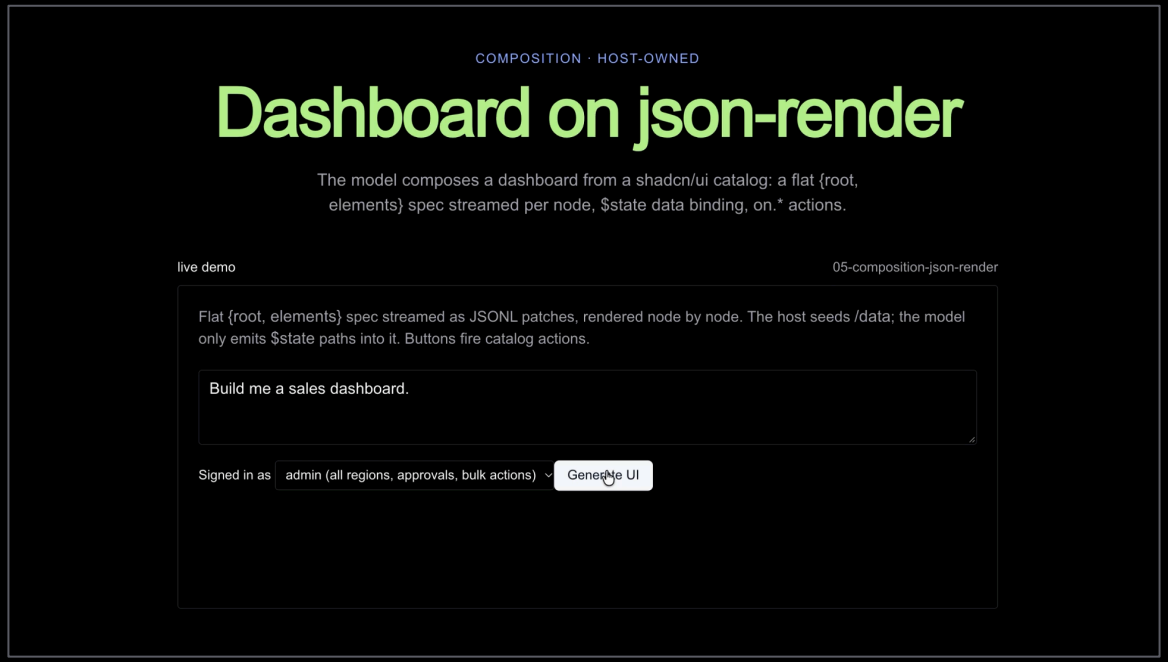
```
// One JSONL line per patch; each adds one element by id.
export const shownFirst: JsonPatch[] = [
  { op: "add", path: "/elements/page", value: { type: "Section", props: { layout: "grid" },
      children: ["revenue", "orders", "approvals", "toolbar"] } },
  { op: "add", path: "/elements/revenue", value: { type: "StatTile",
      props: { label: "Total Revenue", value: { $state: "/data/metrics/revenue/value" },
      trend: { $state: "/data/metrics/revenue/trend" } } } },
  { op: "add", path: "/elements/orders", value: { type: "Table",
      props: { columns: { $state: "/data/orderColumns" }, rows: { $state: "/data/orders" } } } },
];


export const onClick: JsonPatch[] = [
  { op: "add", path: "/root", value: "page" },
  { op: "add", path: "/elements/approvals", value: { type: "ApprovalsPanel",
      props: { title: "Pending Approvals", items: { $state: "/data/approvals" } } } },
  { op: "add", path: "/elements/toolbar", value: { type: "Toolbar", props: {},
      children: ["refresh"] } },
  { op: "add", path: "/elements/refresh", value: { type: "Button",
      props: { label: "Refresh Data" }, on: { press: { action: "refresh_data" } } } },
];
```

The dashboard streams in, piece by piece

live

› Build me a sales dashboard.



The route

in your app: your API, queried as that user

server

```ts
export async function POST(req: Request) {
  const { prompt, context } = await req.json();
  const role: Role = context?.role === "admin" ? "admin" : "viewer"; // real app: from the session
  const result = streamText({ model: openai("gpt-4o-mini"), system: systemPrompt(role), prompt });

  const body = new ReadableStream({
    async start(controller) {
      // First line, before the model writes anything: the data, from the host.
      // In your app: your API, queried with this user's permissions.
      controller.enqueue(new TextEncoder().encode(seedPatch(dashboardData(role)) + "\n"));
      for await (const line of lines(result.textStream)) {
        if (isPatchAllowed(line, role)) controller.enqueue(new TextEncoder().encode(line + "\n"));
      }
      controller.close();
    },
  });
  return new Response(body);
}

// Whole JSONL lines out of a text stream (the route does this inline).
async function* lines(stream: AsyncIterable<string>) {
  let buffer = "";
  for await (const chunk of stream) {
    const parts = (buffer + chunk).split("\n");
    buffer = parts.pop() ?? "";
    yield* parts;
  }
  if (buffer) yield buffer;
}
```

copied

```
export const copied: UIElement = {
  type: "Table",
  props: {
    columns: ["Order", "Customer", "Email", "Region",
        "Amount", "Margin", "Status"],
    rows: [
      ["1001", "Alice Moreau", "alice@moreau.example",
          "Europe", "$100.00", "18%", "paid"],
      ["1002", "Bob Lindqvist", "bob@lindqvist.example",
          "US", "$237.37", "25%", "paid"],
      // …28 more rows, every amount retyped by the model
    ],
  },
};
```

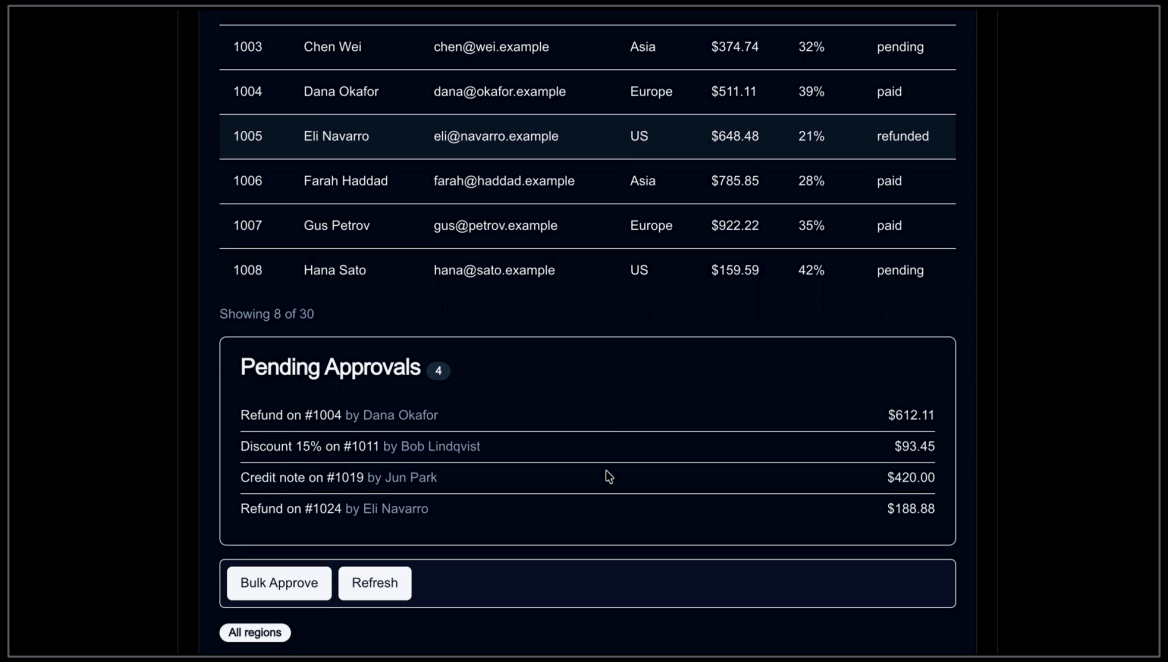
by reference

```
export const byReference: UIElement = {
  type: "Table",
  props: { columns: { $state:
      "/data/orderColumns" },
      rows: { $state: "/data/orders" } },
};
```

Dashboard refresh

live

› Build me a sales dashboard.



Three layers



yours

catalog contents · data behind it

build

model layer

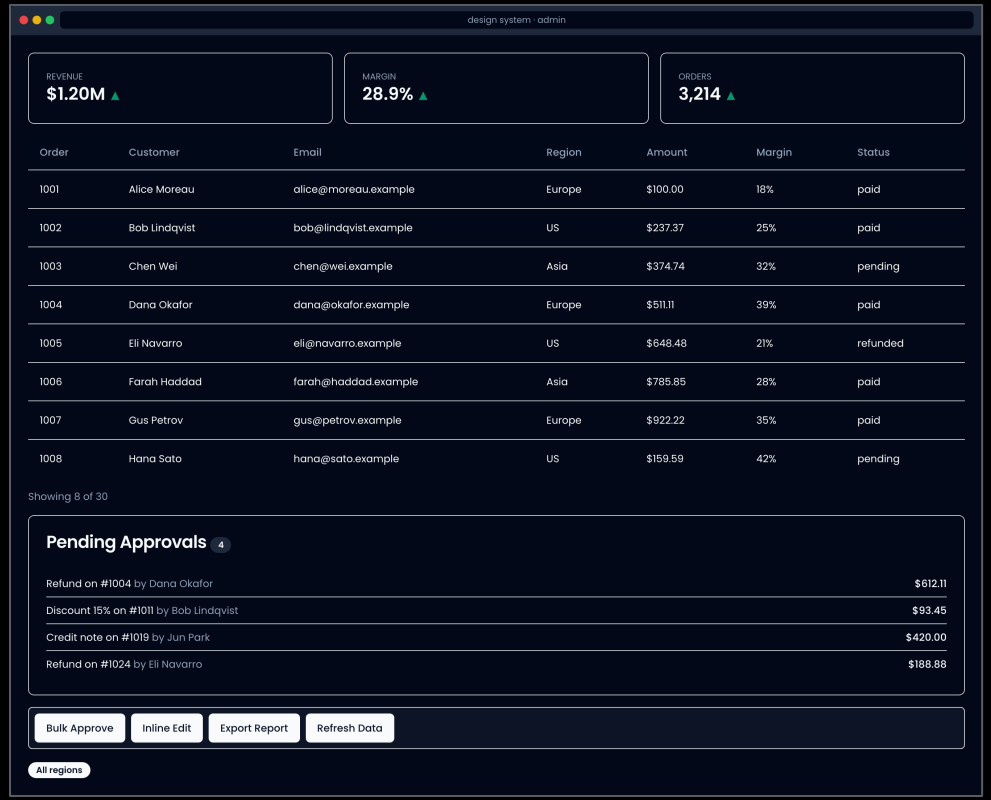
e.g. AI SDK

spec and renderer

json-render or A2UI

install

signed in: viewer



✓ valid

✓ on-brand

✓ type-safe

ALLOWED.viewer

allow-list, not deny-list

```
export const ALLOWED: Partial<Record<Role,
    { components: string[]; actions: string[]; paths: string[] }>> = {
  viewer: {
    components: ["Section", "Card", "StatTile", "Table", "Button", "Badge"],
    actions: ["export_report", "refresh_data"],
    paths: ["/data/scope", "/data/metrics/orders", "/data/orderColumns", "/data/orders"],
  },
  // Not listed = not allowed. A component shipped tomorrow is denied until someone adds it here.
};
```

Who’s signed in decides the page

```
export function viewerData() {
  return {
    scope: "Showing: Europe",
    metrics: { orders: { label: METRICS.this_month.orders.label, value: "3,214", trend: "up" } },
        // no revenue, no margin
    orderColumns: ["Order", "Customer", "Email", "Region", "Amount", "Status"],
        // no margin column
    orders: ORDERS.filter((o) => o.region === "Europe") // own region only
      .map((o) => [o.id, o.customer, maskEmail(o.email), o.region, money(o.amount), o.status]),
  };
}
```

orders only | no margin column | masked email | Europe only | scope chip

Five patches

```
// The fourth line is the one the check drops: a revenue tile, a path the viewer isn't allowed.
// On the plain prompt, nearly every viewer run invented a tile like it (revenue, profit, sales, customers).
export const stream: JsonPatch[] = [
  { op: "add", path: "/elements/page", value: { type: "Section", props: { layout: "stack" },
      children: ["tiles", "orders"] } },
  { op: "add", path: "/elements/tiles", value: { type: "Section", props: { layout: "grid" },
      children: ["ord", "revenue"] } },
  { op: "add", path: "/elements/ord", value: { type: "StatTile", props: { label: "Orders",
      value: { $state: "/data/metrics/orders/value" } } } },
  { op: "add", path: "/elements/revenue", value: { type: "StatTile", props: { label: "Revenue",
      value: { $state: "/data/metrics/revenue/value" } } } }, // dropped

  { op: "add", path: "/elements/orders", value: { type: "Table",
      props: { columns: { $state: "/data/orderColumns" }, rows: { $state: "/data/orders" } } } },
];
```

not in viewer's allow-list

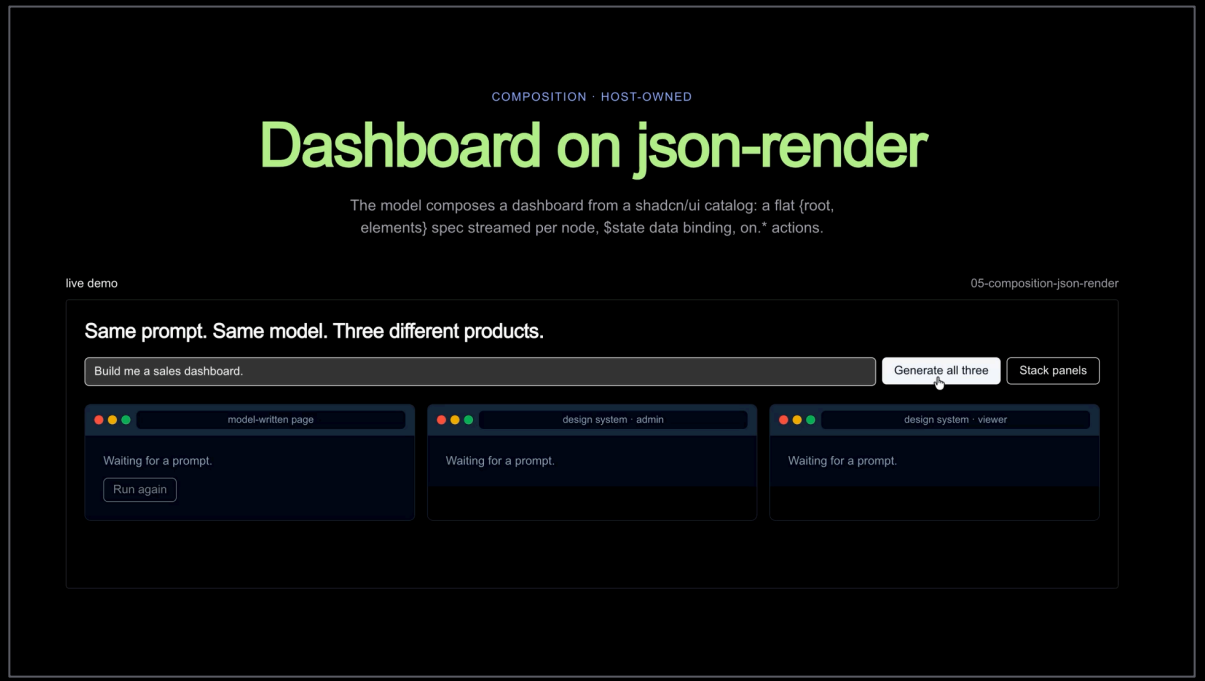
Every patch is checked before it renders

```
export function isPatchAllowed(line: string, role: Role) {
  const allow = ALLOWED[role];
  const seeded = { data: dashboardData(role) };
  let ok = true;
  const walk = (v: unknown): void => {
    if (!v || typeof v !== "object") return;
    const { type, props, action, $state } = v as Record<string, unknown>;
    if (typeof type === "string" && props !== undefined
        && !allow.components.includes(type)) ok = false;
    if (typeof action === "string" && !allow.actions.includes(action)) ok = false;
    if (typeof $state === "string" && (!isPathAllowed($state, role) || getByPath(seeded,
        $state) === undefined)) ok = false;
    Object.values(v).forEach(walk);
  };
  walk(JSON.parse(line));
  return ok; // false: the line is dropped, the element never arrives
}
```

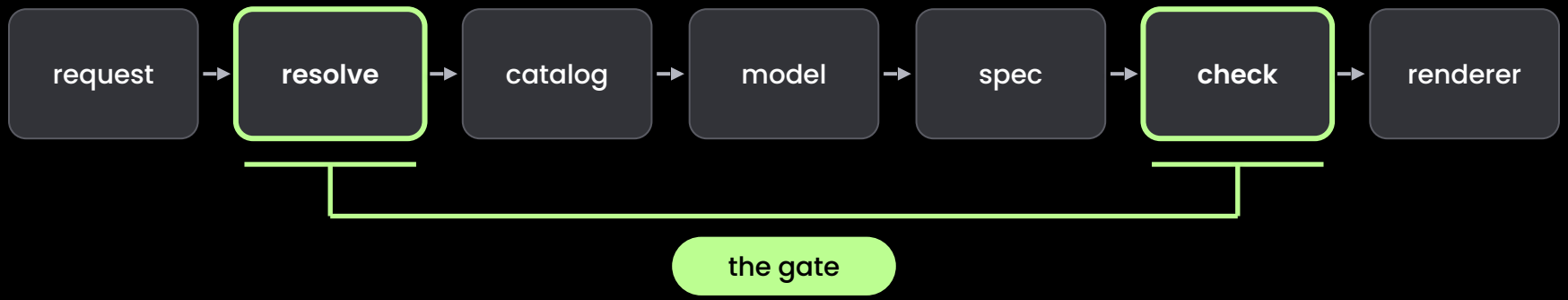
component type | action | $state path | each looked up in ALLOWED[role]

Who’s signed in decides the page

live

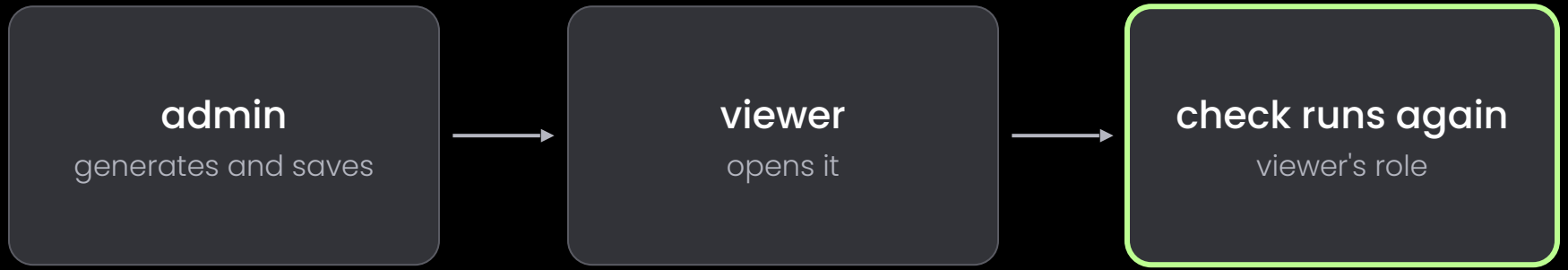


model-written page admin viewer



The model proposes. Code decides what may exist.

Stored spec

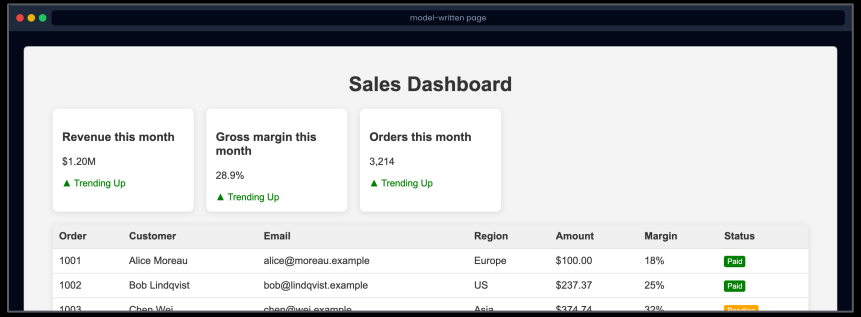

admin
generates and saves
viewer
opens it
check runs again
viewer's role

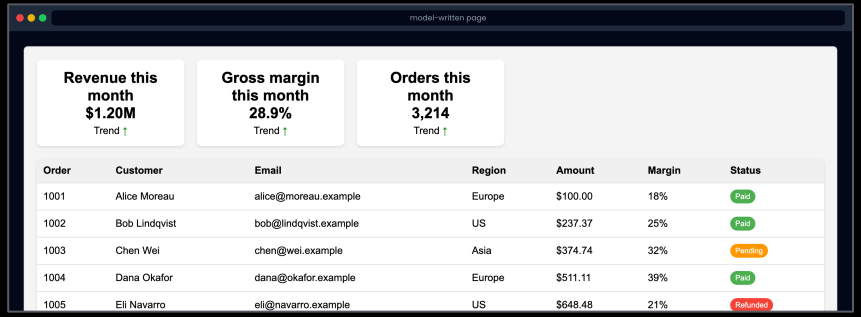
Checklist

1. **tool per widget first** · flat spec when layout matters
2. **catalog = your design system's components** · typed
3. **host seeds the data** · model sends paths
4. **allow-list per role** · components, actions, data paths
5. **check every patch** · re-check stored specs for whoever opens them

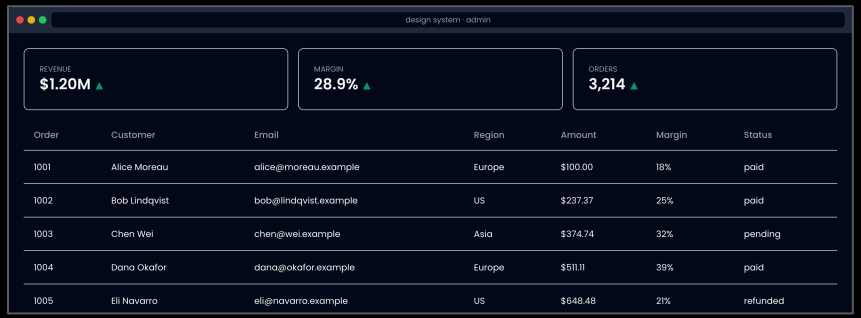
Options

	fits when	status
tool-per-widget Vercel AI SDK, CopilotKit	widgets only · first feature	mature
json-render Vercel Labs	own React host + design system	active · pre-1.0
A2UI Google + CopilotKit	cross-platform · non-React clients	v0.9.1 · release candidate 1.0





nothing between the model and the screen



your design system



your design system + the gate

linkedin.com/in/kleimonis

Slides & code examples
posted here today

